MilaMem Memory System Card

MilaMem is a fictional memory system. This card is a fully populated educational example: every field is filled with plausible but invented values to illustrate how the template is used. It documents no real product.

Contents

Reader's Guide 3

Overview 4

Memory System Details 4

Intended Use 5

Core Memory Capabilities 5

Memory Form and Architecture 6

Memory Lifecycle 6

Performance and Evaluation 7

Limitations and Risks 8

Maintenance and Governance 8

Privacy, Sovereignty, and Safety 9

Citation 10

More Information 10

Reader's Guide

This card is designed to serve multiple stakeholders. Depending on your role, you may want to focus on different sections to find the information most relevant to you.

End Users

End users should focus on sections that explain what the system remembers, how it impacts their experience, and what control they have over their data.

- **Intended Use** — Explains how the memory system is meant to be used directly and outlines any out-of-scope or unsupported uses.
- **Core Memory Capabilities** — Details the exact types of information the system captures, such as user facts, preferences, or task history.
- **Limitations and Risks** — Highlights potential behavioral risks you might encounter, such as false memories, stale information, or inappropriate personalization.
- **Privacy, Sovereignty, and Safety** — Covers critical information about transparency, user control, editability, and how your sensitive data is handled.

Auditors (Compliance & Governance)

Auditors, privacy reviewers, and governance teams should focus on the sections detailing evidence, compliance metrics, and operational safeguards.

- **Overview** — Provides quick, machine-readable metadata regarding the system's evaluation status, privacy review status, and licensing.
- **Performance and Evaluation** — Details the empirical evidence supporting the system's claims, including the evaluation setup, datasets used, and specific performance metrics.
- **Maintenance and Governance** — Outlines audit logging practices, runtime monitoring, user feedback channels, and retention policies.
- **Privacy, Sovereignty, and Safety** — Provides an extended treatment of contextual integrity, PII detection, security threat models, and adversarial robustness.

Developers & System Integrators

Developers and engineers integrating or modifying the system should focus on the technical architecture, implementation details, and lifecycle mechanics.

- **Overview** — Lists the supported integration interfaces (e.g., APIs, SDKs), supported modalities, and physical memory forms.
- **Memory Form and Architecture** — Explains the underlying data representation (such as vector embeddings or graphs), storage locations, and the technical read/write paths.
- **Memory Lifecycle** — Describes the mechanical processes of how information is captured, consolidated, forgotten, and retrieved by the agent or model.
- **Model and Dependency Information** — Lists the required base models, embedding models, databases, and external services necessary to run the system.

Overview

Memory system name: MilaMem
Version: v1.2.0
Developers: MilaMem Project (fictional educational example)
Release date: 2026-03-15
License: Apache-2.0
Homepage: `https://example.org/milamem`
Repository: `https://example.org/milamem/repo`
Technical reference: MilaMem technical report (fictional), 2026
Documentation: `https://example.org/milamem/docs`
Contact: `contact@example.org`
Tags: memory-system | agent-memory | hybrid | vector | graph | open-source
Memory forms: hybrid raw-text; vector embeddings; entity/relation graph
Memory functions: user facts; preferences; experience traces; working context
Modalities: text; structured data
Supported interfaces: API; SDK; MCP server
Evaluation status: internally evaluated (illustrative benchmarks; see Evaluation)
Privacy review status: self-assessed

Memory System Details

System Description

Field	Value
Summary	A hybrid long-term memory layer for LLM agents that extracts salient facts from interactions, stores them as natural-language entries indexed by vector embeddings and a lightweight entity/relation graph, and retrieves them through fused semantic, keyword, and graph signals.
Developed by	MilaMem Project (fictional)
Shared by	Not Applicable (not redistributed by a separate party)
Version	v1.2.0
Release date	2026-03-15
License	Apache-2.0
System type	Memory framework with database backend
Deployment mode	Self-hosted library; optional managed cloud service
Primary users	Developers and agent builders
Primary affected users	End users whose facts and preferences are stored

Sources

Field	Value
Repository	https://example.org/milamem/repo
Documentation	https://example.org/milamem/docs
Technical reference	MilaMem technical report (fictional), 2026
Demo or service endpoint	https://example.org/milamem/demo
Changelog	https://example.org/milamem/changelog

Intended Use

Direct use. MilaMem is intended to add persistent, cross-session memory to chat and tool-using agents through its SDK or API, so that an assistant can recall a user's stated facts and preferences without re-asking.

Downstream use. It may be embedded in larger products such as personal assistants and customer-support bots, or used as a memory backend in research benchmarks for memory-augmented agents.

Out-of-scope use. It is not designed for high-stakes autonomous decisions in medical, legal, or financial settings without human review, nor for storing regulated data without additional controls.

Core Memory Capabilities

Remembered Information

Field	Value
Information types	User facts, preferences, task history, and tool outputs.
Temporal scope	Cross-session, long-term.
Granularity	Atomic facts and short summaries, linked to source messages.
Persistence	Persistent until deletion; optional policy-controlled time-to-live per namespace.

Functional Role

MilaMem enables personalization, factual recall, and task continuity across sessions. In the template's terms it is primarily a *factual memory* (declarative user facts and preferences) and a *retrieval-augmented memory* (external items fetched at inference time), with a modest *experiential memory* component that records task outcomes for light strategy reuse. It does not maintain a dedicated *working memory* for active reasoning, and it is not a *parametric* or *latent* memory: nothing is written into model weights or hidden states.

Capability Boundaries

Remembers well. Stable, explicitly stated user facts and preferences.

Does not remember. Transient working state, non-text modalities, and information the user opts out of storing.

Known failure modes. Stale memories, occasional conflicting facts, and under-retention for heavily paraphrased queries.

Memory Form and Architecture

Representation

Field	Value
Representation form	Raw text plus vector embeddings, with an entity/relation graph over extracted facts.
Human readability	Human-readable: entries are stored as natural-language text.
Addressability	By ID, semantic search, and graph traversal.
Structure	Flat list of entries with an overlaid semantic graph layer.
Storage location	Vector database and graph database (self-hosted or managed).

Agent or Model Coupling

MilaMem is an external component accessed over an API by the agent harness through a memory-manager middleware. Reads are agent-triggered: the agent issues a query and the retrieved entries are injected into the prompt context. Writes are asynchronous at turn boundaries, when extracted facts are deduplicated, merged, and persisted to the vector and graph stores. The memory remains separate from the model, so no weights are modified.

Data Sources

Inputs are drawn from user messages, tool outputs, and agent reasoning traces. Collection is triggered at the end of each turn and on explicit save commands. Before storage, content is summarized, structured into atomic facts, embedded, deduplicated, and passed through PII redaction.

Modality Support

MilaMem supports text and structured records only. Cross-modal alignment is Not Applicable, since image, audio, and video are out of scope for this version.

Memory Lifecycle

Capture or Formation

At the end of a turn, an LLM extractor scores candidate facts against a saliency prompt and keeps salient user facts, preferences, and task outcomes. Selected items are summarized, structured, embedded, and redacted before storage, and each is checked with confidence scoring, deduplication, and schema validation so that low-quality or malformed entries are dropped early.

Maintenance or Evolution

New facts are merged into a matching entry when one exists and appended otherwise, so the store evolves rather than simply growing. Conflicts are resolved by recency and confidence, with authenticated user corrections taking priority over agent-supplied values. A periodic consolidation job merges near-duplicates and summarizes clusters, while time-based expiration and low-utility pruning remove stale entries; manual deletion is always available. Entries carry timestamps and revision counters so prior states remain traceable.

Access, Retrieval, or Use

At task start and on demand during a turn, the agent constructs a query from the current context and MilaMem retrieves candidates through semantic search combined with graph expansion. Candidates are ranked by a fused score over similarity, recency, and graph distance, then re-ranked, filtered, and deduplicated. The result is returned as a small set of ranked natural-language snippets, each carrying its source identifiers so the agent can ground its response.

Performance and Evaluation

Performance Claims

The claimed benefits are improved recall and task success with reduced user repetition. The evidence level is internal testing on public benchmarks, with no external audit; results are reported against a no-memory baseline and a standard retrieval-augmented (RAG) baseline.

Evaluation Setup

Field	Value
Benchmarks or tasks	LoCoMo, LongMemEval, and a small custom suite.
Evaluation level	Component-level and system-level agent evaluation.
Environment	Chat and tool-use settings on offline datasets.
Datasets or traces	Public benchmark traces plus synthetic dialogues; no sensitive data.
Metrics	Accuracy, recall@k, and retrieval latency.
Reproducibility	Code, configs, seeds, and prompts released.

Evaluation Results

Benchmark (metric)	No memory	RAG	MilaMem
LoCoMo (accuracy)	38.0%	61.5%	70.2%
LongMemEval (accuracy)	41.3%	64.0%	73.8%
Custom suite (recall@5)	0.42	0.69	0.81

Illustrative, internally reported results for this fictional example; numbers are invented to demonstrate the card and should not be cited as real measurements.

Memory Quality

Capture quality is high for explicit facts and lower for implied preferences. Update quality is strong on duplicates, with conflicts resolved by recency. Use quality is good for top-5 retrieval but weaker on heavily paraphrased queries. The system is calibrated to return an empty set below a similarity threshold rather than guess, and grounding is supported because every returned snippet links to its source message IDs.

Efficiency and Scalability

Field	Value
Latency	Median retrieval under 150 ms at 100k entries.
Storage footprint	Roughly 2 KB per entry plus index overhead.
Compute cost	Dominated by embedding and periodic LLM extraction.
Scale tested	Up to 1M entries across 10k users.
Degradation at scale	Latency and conflict rate rise gradually past 1M entries.

Utility Trade-offs

The main performance trade-off is accuracy versus retrieval latency, since wider candidate sets and reranking improve recall at higher cost. Operationally, self-hosting offers control and portability at the price of greater maintenance burden compared with the managed service.

Limitations and Risks

Technical limitations. MilaMem handles text and structured data only, and capture quality is bounded by the accuracy of LLM extraction.

Behavioral risks. False or stale memories and inappropriate personalization can arise from over-retention or unresolved conflicts.

Domain and user limitations. It is validated only on English benchmarks; other languages and specialized domains are untested.

Recommendations. Enable PII redaction, monitor retrieval quality, and expose stored memories for user review before deployment.

Maintenance and Governance

Updates

Minor releases ship roughly quarterly. The API and storage schema are stable within a major version, and re-embedding and re-indexing scripts are provided to migrate existing memories across embedding-model or schema changes.

Monitoring

Operators can monitor retrieval quality, latency, write and deletion logs, and privacy events. Read, write, and deletion events are recorded in audit logs accessible to operators only and retained for 90 days. Users can submit corrections, deletion requests, and issue reports.

Model and Dependency Information

Field	Value
Base models	Configurable; defaults to an open instruction-tuned LLM.
Embedding models	Configurable open text-embedding model.
Rerankers or classifiers	Cross-encoder reranker and a PII classifier.
Databases and storage systems	A vector database and a graph database.
External services	Optional managed cloud and object storage for backups.

Privacy, Sovereignty, and Safety

Privacy Definition Used

MilaMem frames privacy through *contextual integrity*: information should flow only in ways appropriate to the context in which it was shared. It also adopts *individual data sovereignty*, giving users visibility and control over their data, and treats the *privacy tax* of persistence explicitly, weighing memory-driven utility against the risk of recombining facts across contexts.

Data Sensitivity

The sensitive categories of concern are PII, location, and private communications. By default such data is redacted before storage unless the user confirms retention. The system can infer preferences and routines from benign inputs, but sensitive inferences are not stored by default.

Transparency

Users can list and search all stored entries, and inspection is complete because every entry is user-visible. Each entry tracks its source message, timestamp, and transformations, and derived entries link back to the entries they came from. There are no hidden profiles or latent state: all memory is inspectable.

User Control, Editability, and Erasability

Users can edit entries directly or through correction requests, and can delete at the entry, session, entity, or account level. Deletion is hard deletion with tombstones, and backups are purged on a fixed cycle. Memory can be exported in full as JSON, and authenticated user corrections override later agent inputs.

Contextual Integrity

MilaMem enforces contextual integrity by ensuring information flows only in ways appropriate to the context in which it was shared. Memory is partitioned into per-user and per-application namespaces, enforced by namespace isolation, access control, and retrieval filters, so that cross-context flows which would violate contextual norms or user expectations are disabled by default and require explicit configuration. Low-trust content from tools or the web is quarantined and cannot write to durable memory without confirmation.

Retention Policy Adherence

Retention can be set by users and administrators. Do-not-remember patterns are blocked before storage, and incognito or temporary sessions are excluded from durable memory. Optional per-namespace time-to-live

drives auto-expiration; backups are retained for 30 days and audit logs for 90.

PII and Sensitive Information Handling

PII is detected with a classifier plus rule-based patterns and, by default, redacted before storage, though the user may confirm storage. After redaction the non-sensitive structure of an entry is retained with sensitive values masked.

Security and Adversarial Robustness

The threat model covers prompt injection, cross-origin extraction, and data poisoning. The internal suite includes injection and deletion-bypass tests. Access controls comprise authentication, authorization, tenant isolation, and encryption at rest, and incident response covers logging, detection, user notification, and rollback.

Privacy Evaluation

Privacy is checked through internal contextual-integrity and retention tests, with no external audit. The main known risk is that stored facts can be recombined across the contexts in which they were shared; mitigations are namespace isolation, redaction, and user inspection and deletion. The residual risk is inference of sensitive traits from aggregated benign facts.

Citation

BibTeX:

```
@techreport{milamem2026,
   title = {MilaMem:  A Hybrid Memory Layer for LLM Agents},
   author = {MilaMem Project},
   year = {2026},
   note = {Fictional educational example}}
```

APA or plain-language citation: MilaMem Project. (2026). *MilaMem: A hybrid memory layer for LLM agents* (fictional educational example).

More Information

Fictional example; see the (placeholder) documentation at `https://example.org/milamem/docs` for the conventions this card illustrates.